UBT
جامعة الأعمال و التكنولوجيا
UNIVERSITY OF BUSINESS AND TECHNOLOGY

JCE | College Of Engineering
كلية الهندسة

SE 420 — Software Maintenance & Evolution

Course Project Report

Modernising a Legacy To-Do List Application

A Practical Study in Reverse Engineering, Refactoring, and Reengineering

Prepared by Team Members

Malik AlNajjar — WA0040

Abdulaziz AlDharrab — YC0003

Instructor: Dr. Ahmed Saeed

Semester: Spring 2026

1. System Description and Literature Review

1.1 System Selected

For this project, the team selected a small command-line To-Do List application written in Python. The application allows a user to manage personal daily tasks through a text-based menu and exposes the following features:

- Add a new task.
- Delete an existing task by identifier.
- Mark a task as completed.
- List all tasks currently stored in memory.

The legacy version of the system was deliberately written in a procedural style with global state, duplicated logic, and a single long function that mixed user input, output, and business rules. This made it a good candidate for a maintenance and evolution exercise, because the problems it exhibits map directly onto the concepts covered in the SE 420 syllabus.

The stated reason for improvement is that the system contains duplicated logic, weak modularity, and poor separation of concerns, which makes it fragile and costly to extend with new features such as task priority.

1.2 Literature Review

Software maintenance is defined as the modification of a software product after delivery to correct faults, improve performance or other attributes, or adapt the product to a changed environment (IEEE Std. 14764). In Pressman's treatment of software engineering, maintenance is presented not as a single activity but as a family of activities including corrective, adaptive, perfective, and preventive maintenance, all of which aim to extend the useful life of a system.

Reverse engineering, as discussed by Chikofsky and Cross, is the process of analysing a subject system to identify its components and their interrelationships, and then creating representations of the system at a higher level of abstraction. In this project, reverse engineering is used to recover a class-level view of the legacy script before any code is modified, so that structural problems can be discussed with evidence rather than intuition.

Refactoring, as formalised by Fowler, is defined as a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behaviour. Fowler's catalogue of code smells — long methods, duplicated code, large classes, and poor naming — provides the vocabulary used in Section 3 of this report to classify the problems found in the legacy system.

These three sources frame the project directly: Pressman supplies the high-level taxonomy of maintenance activities, Chikofsky and Cross justify the reverse-engineering step, and Fowler provides the practical refactoring techniques applied in Section 5.

References

- Pressman, R. S., & Maxim, B. R. (2015). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.
- Chikofsky, E. J., & Cross, J. H. (1990). Reverse Engineering and Design Recovery: A Taxonomy. IEEE Software, 7(1), 13–17.

2. Identification of Code Problems

Three distinct problems were identified in the legacy source file (todo_legacy.py). For each problem, the offending code is shown, the underlying issue is explained in terms of established code smells, and the most suitable modernisation approach is named.

2.1 Problem 1 — Duplicate Code

Code snippet:

```
def add_task():
    desc = input("Enter task description: ")
    t = {}
    t["id"] = len(tasks) + 1
    t["description"] = desc
    t["status"] = "pending"
    tasks.append(t)
    print("Task added.")

def add_urgent_task():
    desc = input("Enter urgent task description: ")
    t = {}
    t["id"] = len(tasks) + 1
    t["description"] = desc
    t["status"] = "pending"
    tasks.append(t)
    print("Urgent task added.")
```

Explanation. The functions add_task() and add_urgent_task() perform exactly the same sequence of operations: read a description, build a dictionary with an id, a description, and a pending status, and append it to the global task list. The only difference is the wording of the prompts and the confirmation message. This is a textbook example of what Fowler calls Duplicated Code, and it violates the DRY

principle. Any future change to the task structure, for example adding a priority field, would need to be made twice and kept in sync manually.

Modernisation approach. Refactoring, specifically Extract Method combined with parameterisation, is used to collapse both functions into a single add_task(description, priority) method.

2.2 Problem 2 — Long Method with Poor Modularity

Code snippet:

```
def do_everything():
    while True:
        print("1-add 2-delete 3-complete 4-list 5-quit")
        c = input("Choice: ")
        if c == "1":
            d = input("desc: ")
            t = {}
            t["id"] = len(tasks) + 1
            t["description"] = d
            t["status"] = "pending"
            tasks.append(t)
        elif c == "2":
            i = int(input("id: "))
            for x in tasks:
                if x["id"] == i:
                    tasks.remove(x)
                    break
        # ... remaining branches for complete, list, quit ...
```

Explanation. The function do_everything() is a Long Method that handles five unrelated responsibilities: displaying the menu, reading user input, dispatching on the user's choice, mutating the task list, and printing results. The method is also tightly coupled to the console because it calls input() and print() directly, which makes the business logic untestable without a human at the keyboard. This mixes the presentation layer and the domain layer into one monolithic procedure.

Modernisation approach. Restructuring (a form of reengineering) is applied. The business logic is extracted into a TaskManager class, and the menu loop is kept in a thin main.py that deals only with input and output.

2.3 Problem 3 — Global Mutable State

Code snippet:

```
tasks = []   # module-level global list

def add_task():
    ...
    tasks.append(t)  # mutates global directly
```

Explanation. The list tasks is declared at module scope and every function mutates it directly. Global mutable state makes the code difficult to reason about because any function in the file can silently change the state of the system, and it makes automated testing awkward because tests cannot be isolated from one another. Fowler classifies this under Data Class and, more broadly, as a violation of encapsulation.

Modernisation approach. Refactoring through Encapsulate Field: the task collection and the id counter are moved behind the TaskManager class as private attributes, and all mutation happens through explicit methods.

3. Reverse Engineering

To understand the structure of the legacy system before modifying it, the code was analysed and recovered as a class diagram that describes the intended post-refactoring architecture. Producing the diagram during the analysis phase served two purposes: it made the hidden structure of the procedural legacy file explicit, and it gave the team a target design to work towards during the refactoring phase.

The diagram shows two classes. TaskManager is responsible for managing the collection of tasks and for exposing the operations that the application supports. Task is a plain entity that carries the data describing an individual task, together with a small behaviour method for state transition. The association between them is one-to-many, because a single TaskManager may hold any number of Task instances.

Figure 1. Class Diagram — Refactored To-Do List Application

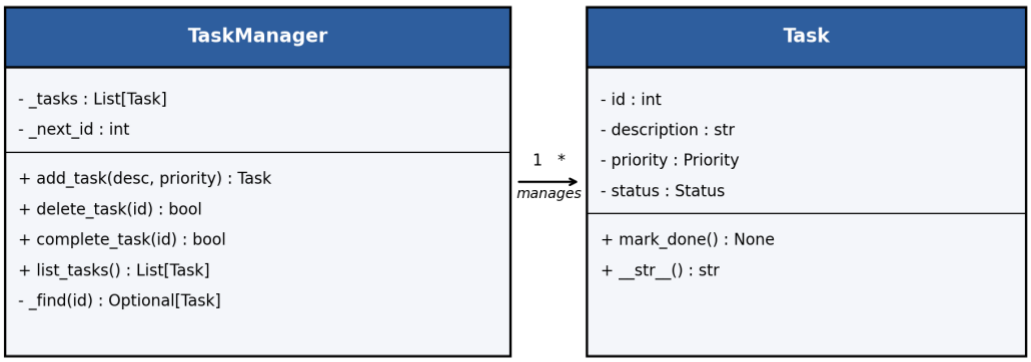

Figure 1 above summarises the recovered design. Compared with the legacy script, which had no classes at all and relied on a module-level list and a long procedural function, this recovered view makes the responsibilities of the system visible and testable.

4. Refactoring Implementation

Four refactoring improvements were applied to the legacy system, each addressing a distinct code smell identified in Section 2. All four preserve the observable behaviour of the application while improving its internal structure, which is the defining property of a refactoring.

4.1 Refactoring 1 — Remove Duplication

Before:

```
def add_task():
    desc = input("Enter task description: ")
    t = {}
    t["id"] = len(tasks) + 1
    t["description"] = desc
    t["status"] = "pending"
    tasks.append(t)
    print("Task added.")

def add_urgent_task():
    desc = input("Enter urgent task description: ")
    t = {}
    t["id"] = len(tasks) + 1
    t["description"] = desc
    t["status"] = "pending"
    tasks.append(t)
    print("Urgent task added.")
```

After:

```
def add_task(self, description: str,
             priority: Priority = "Medium") -> Task:
    task = Task(id=self._next_id,
                description=description,
                priority=priority)
    self._tasks.append(task)
    self._next_id += 1
    return task
```

Explanation. The two near-identical functions are collapsed into a single add_task method on TaskManager that takes the description and priority as parameters. The construction of the task is delegated to the Task dataclass, which removes the repeated dictionary-building code. The resulting

method has a single reason to change, and adding a new attribute to a task now requires a change in exactly one place.

4.2 Refactoring 2 — Extract Method

Before:

```
def do_everything():
    while True:
        print("1-add 2-delete 3-complete 4-list 5-quit")
        c = input("Choice: ")
        if c == "1":
            d = input("desc: ")
            t = {}
            t["id"] = len(tasks) + 1
            t["description"] = d
            t["status"] = "pending"
            tasks.append(t)
        elif c == "2":
            i = int(input("id: "))
            for x in tasks:
                if x["id"] == i:
                    tasks.remove(x)
                    break
        # ... etc ...
```

After:

```
# main.py — only handles I/O
def run() -> None:
    manager = TaskManager()
    while True:
        print(MENU)
        choice = input("Choice: ").strip()
        if choice == "1":
            desc = input("Description: ").strip()
            priority = _read_priority()
            manager.add_task(desc, priority)
        elif choice == "2":
            manager.delete_task(int(input("Task ID: ")))
        elif choice == "3":
            manager.complete_task(int(input("Task ID: ")))
        elif choice == "4":
            for t in manager.list_tasks():
                print(t)
        elif choice == "5":
            break
```

Explanation. The long procedural loop is replaced by a thin CLI in main.py that only reads input and prints output. Every business rule — creating, deleting, completing, and listing tasks — is delegated to TaskManager. This applies Fowler's Extract Method refactoring and produces a cleaner separation between the presentation layer and the domain layer. The TaskManager class is now independent of the console and can be exercised by unit tests without user interaction.

4.3 Refactoring 3 — Rename Variables

Before:

```
c = input("Choice: ")
d = input("desc: ")
i = int(input("id: "))
for x in tasks:
    if x["id"] == i:
```

After:

```
choice = input("Choice: ").strip()
description = input("Description: ").strip()
task_id = int(input("Task ID: "))
for task in self._tasks:
    if task.id == task_id:
```

Explanation. The legacy code used single-letter variable names such as c, d, i, t, and x, which carried no information about what each value represented. A reader had to trace the variable back to its input() call to understand its meaning, and the loop variable x was especially misleading because it obscured that each element was a task. The refactoring renames these variables to choice, description, task_id, and task, which makes the code self-documenting. This is Fowler's Rename Variable refactoring, and it addresses the code smell of poor naming identified in Section 2. The external behaviour of the program is unchanged; only the readability of the source is improved.

4.4 Refactoring 4 — Simplify Long Method

Before:

```
def delete_task(self, task_id):
    found = False
    for task in self._tasks:
        if task.id == task_id:
            self._tasks.remove(task)
            found = True
            break
    if not found:
        raise ValueError("Task not found")
```

After:

```
def delete_task(self, task_id: int) -> None:
    task = self._find(task_id)
    self._tasks.remove(task)

def _find(self, task_id: int) -> Task:
    for task in self._tasks:
        if task.id == task_id:
            return task
    raise ValueError("Task not found")
```

Explanation. The original delete_task method mixed three concerns in one body: iterating over tasks, tracking whether a match was found through a boolean flag, and raising an error when the task was missing. This is a Long Method smell, even though the line count is modest, because the single method handled multiple responsibilities and relied on control-flow bookkeeping. The refactoring extracts the lookup logic into a dedicated private helper _find, which returns the task directly or raises an error if no match exists. The main delete_task method is now two lines and reads as a straightforward statement of intent: find the task, remove it. The same _find helper is reused by complete_task, removing another source of duplicated iteration logic. This refactoring applies both Fowler's Simplify Method and Extract Method patterns and produces code that is easier to read, easier to test, and easier to extend.

5. Reengineering Improvement

Beyond the four behaviour-preserving refactorings, the team implemented one reengineering improvement that adds new functionality to the system. This distinguishes reengineering from refactoring: reengineering changes the observable behaviour of the application, in this case by giving the user a richer vocabulary for describing tasks.

5.1 Improvement — Task Priority Levels

Before. In the legacy system, a task had only a description and a status. There was no way for the user to express urgency, which meant the listing of tasks gave equal weight to a reminder to buy groceries and a deadline to submit an assignment.

After. Every task now carries a priority attribute that takes one of three values: Low, Medium, or High. The priority is captured at task creation time through a short prompt in the command-line interface. The Medium priority is the default when the user accepts the prompt without selecting an option.

Relevant code after the improvement:

```
# task.py
@dataclass
class Task:
    id: int
```

```
    description: str
    priority: Priority = "Medium"   # new attribute
    status: Status = "pending"

    def mark_done(self) -> None:
        self.status = "done"

# task_manager.py
def add_task(self, description: str,
             priority: Priority = "Medium") -> Task:
    task = Task(id=self._next_id,
                description=description,
                priority=priority)
    self._tasks.append(task)
    self._next_id += 1
    return task
```

Justification. The priority attribute provides an explicit extension point for future features that were out of scope for this project but are now straightforward to implement, for example sorting the list of tasks by priority or filtering the list to show only high-priority items. In the legacy code, these features would have required modification in multiple places; in the refactored design they require changes only inside the TaskManager class. This demonstrates that the refactoring step has genuinely improved the modifiability of the system, which is one of the principal goals of software maintenance.

6. Team Collaboration

The workload was divided across two members. Each member led the parts of the project that matched their strengths, and both reviewed every deliverable before submission. Work was coordinated through shared version control, and decisions about naming, structure, and diagram notation were made collectively before being implemented.

Member	Role	Contribution
Malik AlNajjar	Code Analysis, Problem Identification & Literature Review	Read the legacy source file, identified the three problems documented in Section 2, and classified each using Fowler's catalogue. Collected the three references cited in Section 1, prepared the literature review, assembled the final report, and coordinated preparation for the viva.
Abdulaziz AlDharrab	Reverse Engineering, Refactoring & Reengineering	Recovered the class-level structure of the target system and produced the class diagram in Section 3 with the accompanying narrative. Implemented the refactored task.py, task_manager.py, and main.py modules, applied the four refactorings described in Section 4, and added the priority feature described in Section 5.

In practice, the work was not strictly sequential. The problem identification work informed the shape of the class diagram, which in turn constrained the refactoring choices made during implementation. The literature review and report assembly ran in parallel as each section became stable. The team met regularly to synchronise progress and to resolve points of disagreement about the scope of the reengineering improvement.

7. Conclusion

The project demonstrates a practical application of the core concepts from SE 420. The legacy To-Do List application was analysed using reverse engineering, its structural problems were classified using an established catalogue of code smells, and two refactoring techniques were applied to improve the internal structure of the code without changing its observable behaviour. A further reengineering step extended the system with a new priority feature that was only feasible after the refactoring had cleaned up the underlying structure. The resulting system is smaller, more modular, and easier to extend than the version from which the team started.